

ACTIVIDAD

2

Gerardo Gaytan Alvarez

Edgar Ivan Patricio Aizpuro

Documento tecnico y de pruebas

Introduccion

Este documento describe el funcionamiento del programa Centro de operaciones, desarrollado a partir de una Stack (pila) y una Queue (cola) propias, implementadas mediante nodos y referencias (clases Node, Stack y Queue), sin usar `java.util.Stack`, `java.util.Queue` ni `ArrayList`.

Las pruebas se realizaron ejecutando el programa desde la consola y navegando el menu principal, registrando acciones y tareas, y comparando la salida impresa por el programa contra el resultado esperado. Se cubren tanto casos normales de operacion como los casos limite solicitados en la actividad (pila vacia y cola vacia).

Funcionamiento de la Stack (Pila)

La Stack esta formada por nodos enlazados y utiliza una sola referencia, `top`, que siempre apunta al ultimo nodo agregado. `push()` crea un nuevo nodo, conecta su `next` hacia el `top` actual y despues mueve `top` hacia el nuevo nodo, dejandolo en la parte superior. `pop()` primero valida con `isEmpty()` si la pila tiene elementos; si los tiene, guarda el data del nodo `top`, mueve `top` a `top.next` y regresa el dato guardado. `peek()` funciona igual que `pop()` pero sin mover `top`, solo lee el dato del nodo superior. `isEmpty()` regresa `true` cuando `top` es `null`, y `size()` recorre la lista desde `top` hasta `null` contando los nodos.

Como `top` siempre se mueve hacia el ultimo nodo agregado, el primer elemento que sale con `pop()` es siempre el ultimo que entro. Este comportamiento es el que se conoce como LIFO (Last In, First Out).

Funcionamiento de la Queue (Cola)

La Queue tambien esta formada por nodos enlazados, pero utiliza dos referencias: `front`, que apunta al primer nodo (el mas antiguo), y `rear`, que apunta al ultimo nodo agregado. `enqueue()` crea un nuevo nodo; si la cola esta vacia, `front` y `rear` apuntan al nuevo nodo, y si ya tiene elementos, se conecta `rear.next` hacia el nuevo nodo y despues `rear` se mueve hacia el. `dequeue()` valida con `isEmpty()` si hay elementos; si los hay, guarda el data del nodo `front`, mueve `front` a `front.next`, y si `front` queda en `null` tambien pone `rear` en `null`. `peek()` lee el data de `front` sin moverlo. `isEmpty()` regresa `true` cuando `front` es `null`, y `size()` recorre la lista desde `front` hasta `null` contando los nodos.

Como los elementos se agregan siempre por `rear` y se retiran siempre por `front`, el primer elemento que entra es siempre el primero en salir. Este comportamiento es el que se conoce como FIFO (First In, First Out).

Manejo de casos limite

Tanto `pop()` y `peek()` en la Stack, como `dequeue()` y `peek()` en la Queue, verifican primero con `isEmpty()` si la estructura tiene nodos. Si esta vacia, el metodo regresa `null` en lugar de intentar leer un nodo que no existe. En la clase `Main`, antes de llamar a estos metodos siempre se pregunta primero con `isEmpty()` y se muestra un mensaje al usuario indicando que no hay acciones o tareas disponibles, evitando que el programa se detenga por una excepcion.

Casos de prueba - Stack

Pruebas de las operaciones basicas de la pila: push, peek, pop, size, isEmpty, y el manejo de una pila vacia.

#	Prueba	Entrada	Resultado esperado	Resultado obtenido
1	Agregar la primera accion	push("Crear archivo")	La pila pasa a tener un nodo (top apunta a "Crear archivo")	Correcto
2	Agregar mas acciones	push("Editar archivo"), push("Renombrar archivo")	La pila queda, de top a fondo: Renombrar archivo, Editar archivo, Crear archivo	Correcto
3	Ver ultima accion	peek()	Devuelve "Renombrar archivo" sin eliminarla	Correcto: "Renombrar archivo"
4	Deshacer ultima accion	pop()	Devuelve y elimina "Renombrar archivo"; top pasa a "Editar archivo"	Correcto: "Renombrar archivo"
5	Ver ultima accion (de nuevo)	peek()	Devuelve "Editar archivo"	Correcto: "Editar archivo"
6	Tamano de la pila	size()	Devuelve el numero de nodos restantes	Correcto: 2
7	Verificar si esta vacia	isEmpty()	Devuelve false, pues hay acciones registradas	Correcto: false
8	Pila vacia - pop	pop() sobre una pila sin nodos	Devuelve null, sin excepcion	Correcto: null
9	Pila vacia - peek	peek() sobre una pila sin nodos	Devuelve null, sin excepcion	Correcto: null
10	Pila vacia - isEmpty	isEmpty() sobre una pila sin nodos	Devuelve true	Correcto: true

Casos de prueba - Queue

Pruebas de las operaciones basicas de la cola: enqueue, peek, dequeue, size, isEmpty, y el manejo de una cola vacia.

#	Prueba	Entrada	Resultado esperado	Resultado obtenido
1	Agregar la primera tarea	enqueue("Tarea 1")	La cola pasa a tener un nodo (front y rear apuntan a "Tarea 1")	Correcto
2	Agregar mas	enqueue("Tarea 2"),	La cola queda, de front a	Correcto

#	Prueba	Entrada	Resultado esperado	Resultado obtenido
	tareas	enqueue("Tarea 3")	rear: Tarea 1, Tarea 2, Tarea 3	
3	Ver siguiente tarea	peek()	Devuelve "Tarea 1" sin eliminarla	Correcto: "Tarea 1"
4	Procesar tarea	dequeue()	Devuelve y elimina "Tarea 1"; front pasa a "Tarea 2"	Correcto: "Tarea 1"
5	Ver siguiente tarea (de nuevo)	peek()	Devuelve "Tarea 2"	Correcto: "Tarea 2"
6	Tamano de la cola	size()	Devuelve el numero de nodos restantes	Correcto: 2
7	Verificar si esta vacia	isEmpty()	Devuelve false, pues hay tareas pendientes	Correcto: false
8	Cola vacia - dequeue	dequeue() sobre una cola sin nodos	Devuelve null, sin excepcion	Correcto: null
9	Cola vacia - peek	peek() sobre una cola sin nodos	Devuelve null, sin excepcion	Correcto: null
10	Cola vacia - isEmpty	isEmpty() sobre una cola sin nodos	Devuelve true	Correcto: true

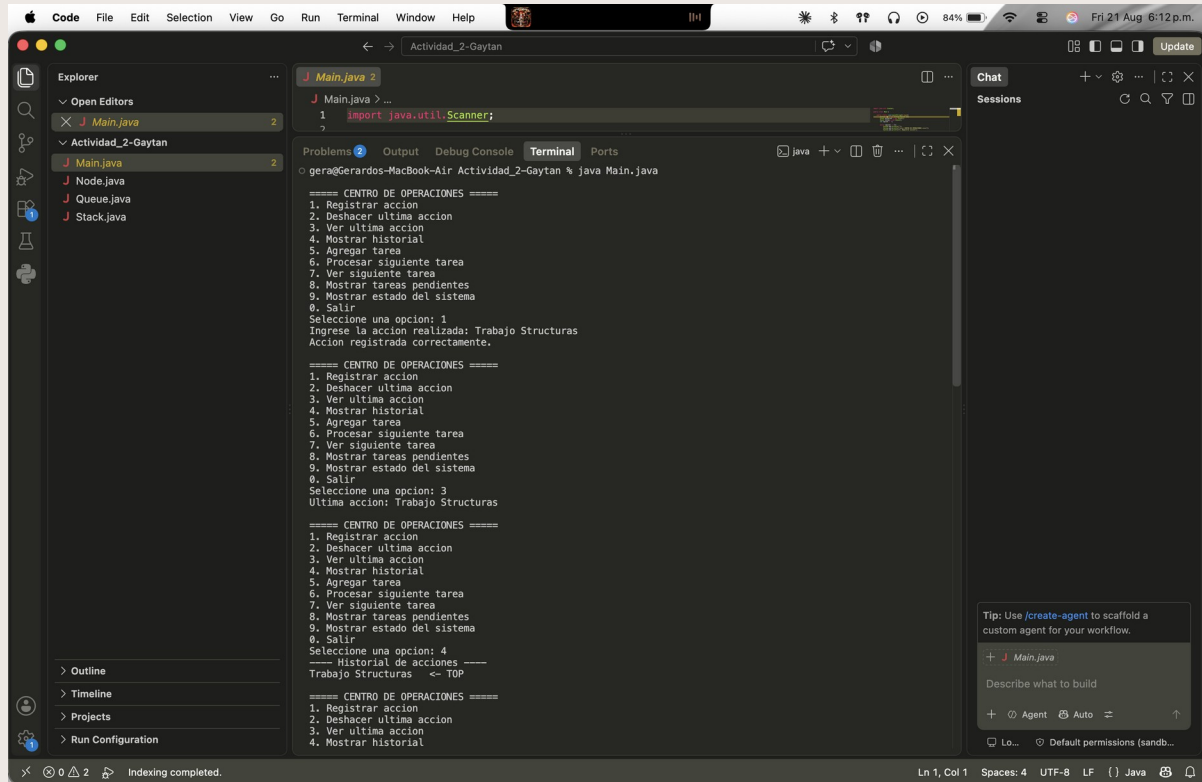
Prueba de comportamiento: LIFO vs FIFO

Prueba donde se agregan los mismos elementos (A, B, C, D) a la Stack y a la Queue, para demostrar claramente la diferencia de comportamiento entre ambas estructuras.

#	Prueba	Entrada	Resultado esperado	Resultado obtenido
1	Stack: agregar elementos	push(A), push(B), push(C), push(D)	La pila queda, de top a fondo: D, C, B, A	Correcto
2	Stack: retirar elementos	pop() cuatro veces	Se retiran en el orden D, C, B, A (LIFO)	Correcto: D, C, B, A
3	Queue: agregar elementos	enqueue(A), enqueue(B), enqueue(C), enqueue(D)	La cola queda, de front a rear: A, B, C, D	Correcto
4	Queue: retirar elementos	dequeue() cuatro veces	Se retiran en el orden A, B, C, D (FIFO)	Correcto: A, B, C, D

Evidencia de ejecucion

A continuacion se incluyen las capturas de pantalla que evidencian el funcionamiento del programa: registrar una accion, ver la ultima accion, mostrar el historial marcando el TOP, agregar una tarea, procesar la siguiente tarea, y el manejo de una cola de tareas vacia.



```
1 import java.util.Scanner;
```

```
gera@Gerardos-MacBook-Air Actividad_2-Gaytan % java Main.java
```

```
===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 1
Ingrese la accion realizada: Trabajo Estructuras
Accion registrada correctamente.

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 3
Ultima accion: Trabajo Estructuras

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 4
----- Historial de acciones -----
Trabajo Estructuras  <- TOP

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
```

Registrar accion, ver ultima accion y mostrar historial marcando el TOP.

```
J Main.java 2
1 import java.util.Scanner;
```

```
gera@Gerardos-MacBook-Air Actividad_2-Gaytan % java Main.java
===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 5
Ingrese la tarea: Tarea base de datos
Tarea agregada correctamente.

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 4
----- Historial de acciones -----
Trabajo Estructuras <- TOP

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 6
Procesando: Tarea base de datos

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
```

Agregar tarea, mostrar historial y procesar la siguiente tarea.

```
J Main.java 2
1 import java.util.Scanner;
```

```
gera@Gerardos-MacBook-Air Actividad_2-Gaytan % java Main.java
===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 8
No hay tareas pendientes.

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: 9
----- Estado del sistema -----
Acciones en el historial: 1
Historial vacio: false
Tareas pendientes: 0
Tareas vacias: true

===== CENTRO DE OPERACIONES =====
1. Registrar accion
2. Deshacer ultima accion
3. Ver ultima accion
4. Mostrar historial
5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Mostrar estado del sistema
0. Salir
Seleccione una opcion: █
```

Manejo de la cola de tareas vacia y despliegue del estado del sistema.

Conclusion

Las pruebas descritas en este documento se ejecutaron sobre el programa compilado y se obtuvo en cada caso el resultado esperado, sin errores ni excepciones. El programa respeta correctamente el comportamiento LIFO de la Stack y el comportamiento FIFO de la Queue, y maneja de forma adecuada los casos de pila vacía y cola vacía.